

Apunte de Cátedra – Computación II

Clase 5: Medición de Rendimiento Empírico

Unidad 2 y Laboratorio 1: De la Teoría a la Práctica

Año 2025 – Modalidad Híbrida
Prof. Marcelo Fabio Roldán
Departamento DACEFyN – Sede Capital – UNLaR

1. Pregunta Disparadora: ¿Por qué mi algoritmo $O(n)$ es más lento que uno $O(n \log n)$?

Imagina que implementas un algoritmo de búsqueda lineal $O(n)$ y tu compañero implementa búsqueda binaria $O(\log n)$. Según la teoría, su algoritmo debería ser mucho más rápido. Pero cuando lo pruebas con 50 elementos... ¡el tuyo es más rápido!

¿Por qué?

💬 "La teoría nos dice qué algoritmo debería ser más rápido. El profiling nos dice cuál realmente lo es."

Este fenómeno es común y revela una verdad fundamental: el rendimiento teórico no siempre se traduce directamente en rendimiento real.

2. Rendimiento Teórico vs. Rendimiento Real

Aspecto	Rendimiento teórico	Rendimiento real
Definición	Comportamiento asintótico según análisis matemático.	Tiempo de ejecución medido en hardware específico.
Enfoque	Escalabilidad a gran escala ($n \rightarrow \infty$).	Comportamiento con datos reales y hardware específico.
Herramientas	Notaciones asintóticas (O , Ω , Θ).	Profilers, cronómetros, benchmarks.
Limitaciones	Ignora constantes, caché, overhead.	Depende del sistema, compilador, datos.

Ejemplo: Merge Sort vs. Bubble Sort

- Teoría:
 - Merge Sort: $O(n \log n)$
 - Bubble Sort: $O(n^2)$
→ Merge Sort siempre gana.
- Realidad (n = 50):
 - Merge Sort: alto overhead de recursión y memoria adicional.
 - Bubble Sort: simple, con buena localidad de caché.
→ Bubble Sort puede ser más rápido.

🔍 Conclusión:

- Para n pequeño: algoritmos simples pueden ser más rápidos.
- Para n grande: el algoritmo con mejor complejidad asintótica domina.
- La transición ocurre en un "punto de quiebre" que depende del hardware y los datos.

3. Factores que Impactan el Rendimiento en Sistemas Reales

El rendimiento real no depende solo del algoritmo, sino de múltiples capas del sistema:

Factor	Descripción	Impacto
Constantes Multiplicativas	Un algoritmo $O(n)$ con constante 1000 es más lento que uno $O(n \log n)$ con constante 10 para n pequeño.	Puede invalidar ventajas asintóticas.
Localidad de Memoria	Acceder a datos secuencialmente (buena localidad) es mucho más rápido que accesos aleatorios.	Diferencias de 100x en velocidad.
Overhead de Recursión	Cada llamada recursiva consume espacio en la pila y tiempo de gestión.	Merge Sort sufre con arrays pequeños.
Optimizaciones del Compilador	El compilador puede desenrollar bucles, eliminar código muerto, etc.	Puede acelerar código significativamente.
Jerarquía de Caché	L1, L2, L3, RAM, disco. Un <i>cache miss</i> puede costar 100ns.	Algoritmos con buena localidad son más

Factor	Descripción	Impacto
		rápidos.
Sistema Operativo	Interrupciones, procesos en segundo plano, gestión de memoria.	Puede introducir <i>noise</i> en mediciones.

🌐 Caso real: En 2023, Google descubrió que un algoritmo $O(n)$ en su motor de búsqueda era más lento que uno $O(n \log n)$ debido a *cache misses*. La solución: reorganizar la estructura de datos para mejorar la localidad espacial.

4. Introducción al Profiling

El profiling es la técnica de medir empíricamente el rendimiento de un programa para identificar cuellos de botella (bottlenecks).

💬 "El profiling no es opcional. Es la diferencia entre adivinar dónde está el problema y saberlo con certeza."

¿Qué Mide un Profiler?

Métrica	Descripción
Tiempo de Ejecución	Cuánto tiempo tarda cada función.
Número de Llamadas	Cuántas veces se llama a cada función.
Uso de Memoria	Cuánta memoria asigna y libera el programa.
Accesos a Disco/Red	Operaciones costosas que ralentizan el sistema.
Cache Misses	Cuántas veces el CPU no encuentra datos en caché.

5. Herramientas de Profiling

a) Profilers Generales

Herramienta	Plataforma	Tipo	Uso
Intel VTune	Windows/Linux	CPU, GPU,	Profiling avanzado, usado por Intel,

Herramienta	Plataforma	Tipo	Uso
		FPGA	Microsoft.
gprof	Linux	CPU	Profiling clásico, basado en muestreo.
Valgrind (Callgrind)	Linux	CPU, Memoria	Detecta <i>memory leaks</i> y cuellos de botella.

b) Bibliotecas de Timing

Lenguaje	Biblioteca	Uso
Python	time, cProfile, line_profiler	Medir funciones, líneas, módulos.
C++	chrono	Alta precisión, ideal para benchmarking.
Haskell	criterion	Benchmarking estadístico, promedios robustos.

c) Profiling en Sistemas Distribuidos

Herramienta	Uso
Jaeger	Tracing distribuido, usado en microservicios.
Zipkin	Monitoreo de latencia entre servicios.
Prometheus + Grafana	Métricas en tiempo real.

✂️ Consejo del Profesor:

"VTune es el Ferrari de los profilers. Si aprenden a usarlo, pueden trabajar en cualquier empresa de software serio del mundo."

6. Metodología de Profiling: Medir Primero, Optimizar Después

Paso 1: Definir el Problema

- ¿El programa es lento en general?
- ¿Solo en ciertas operaciones?
- ¿Con ciertos tipos de datos?

Paso 2: Seleccionar Herramienta y Datos

- Usa datos representativos del uso real.
- Ejecuta múltiples iteraciones y toma el promedio.
- Asegúrate de que el sistema esté caliente (*warm-up*): ejecuta el código una vez antes de medir.

Paso 3: Ejecutar el Profiler

- En Python:

```
python
1 import cProfile
2 cProfile.run('mi_funcion(n=10000)')
```

Paso 4: Interpretar Resultados

Ejemplo de Salida de cProfile

	ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1						
2	1	0.000	0.000	2.500	2.500	busqueda_lineal.py:5(lineal)
3	1	0.000	0.000	0.001	0.001	busqueda_binaria.py:8(binaria)

- ncalls: Número de llamadas.
- tottime: Tiempo exclusivo de la función.
- cumtime: Tiempo total (incluye subfunciones).
- percall: Tiempo promedio por llamada.

🔍 **Clave:** Si una función tiene alto cumtime y muchas ncalls, es un *bottleneck*.

7. Laboratorio Práctico: Profiling con Herramientas Reales

Objetivo

Aplicar técnicas de profiling para identificar y optimizar cuellos de botella en algoritmos.

Tareas

Ejercicio 1: Comparar Búsqueda Lineal vs. Binaria

1. Implementa ambas búsquedas en Python.
2. Mide el tiempo con time para tamaños: 1K, 10K, 100K, 1M.

3. Usa cProfile para analizar el rendimiento.
4. Grafica los resultados y encuentra el punto de quiebre.

Ejercicio 2: Optimizar un Algoritmo Ineficiente

1. Dado un código con búsqueda lineal $O(n)$ en un bucle $O(n) \rightarrow O(n^2)$.
2. Usa line_profiler para identificar la línea crítica.
3. Reemplaza con un dict (hashing) $\rightarrow O(1)$.
4. Mide la mejora: ¿2500x más rápido?

Ejercicio 3: Profiling en Haskell (TP3)

1. Usa criterion para benchmarkear una función recursiva.
2. Compara con versión optimizada (memoización).
3. Documenta la mejora.

8. Casos de Estudio: Antes y Después

Caso 1: Búsqueda Ineficiente

- Antes: Búsqueda lineal $O(n) \rightarrow 2.5$ segundos.
- Después: HashMap (diccionario) $O(1) \rightarrow 0.001$ segundos.
- Mejora: 2500x más rápido.

Caso 2: Cache Misses

- Antes: Accesos aleatorios a matriz $\rightarrow$ alto *cache miss rate*.
- Después: Recorrido por filas (localidad espacial) $\rightarrow$ mejora de 10x.

Caso 3: Recursión Innecesaria

- Antes: Fibonacci recursivo $\rightarrow O(2^n)$, 30 segundos.
- Después: Iterativo con DP $\rightarrow O(n)$, 0.001 segundos.
- Mejora: 30,000x más rápido.



Lección:

"La optimización prematura es el origen de todos los males" (Donald Knuth). Pero la optimización basada en datos es esencial."

9. Mejores Prácticas (Best Practices)

- ☑ Compila con optimizaciones habilitadas (-O2 en C++, -O en Haskell).
- ☑ Usa datos representativos del uso real.
- ☑ Ejecuta múltiples iteraciones y toma el promedio.
- ☑ Aísla procesos que interfieran (otros programas, antivirus).
- ☑ Documenta cambios y mediciones para comparar versiones.
- ☑ No optimices código no crítico: el 90% del tiempo se pasa en el 10% del código (Ley de Pareto).

☐ Errores Comunes

- ☑ Usar datasets demasiado pequeños.
- ☑ No considerar el *warm-up* del sistema.
- ☑ Optimizar código antes de medir.
- ☑ Ignorar la corrección después de optimizar.

10. Conexión con los Trabajos Prácticos

- TP1 (Análisis Asintótico): Ahora puedes validar empíricamente tus predicciones teóricas.
- TP3 (Programación Funcional en Haskell): Usa criterion para benchmarkear tus funciones recursivas y comparar con versiones optimizadas.

★ Consejo del Profesor:

"La tarea que les asigno no es solo ejercicio académico. Los patrones que descubran — cómo el rendimiento escala con el tamaño de input, cómo diferentes implementaciones pueden tener rendimientos muy diferentes — son insights que usarán durante toda su carrera."

11. Conceptos Clave Aprendidos

- ☑ El rendimiento teórico (asintótico) no siempre predice el rendimiento real.
- ☑ Factores como constantes, caché, overhead y compilador impactan fuertemente el rendimiento.
- ☑ El profiling es esencial para identificar *bottlenecks* reales.
- ☑ Herramientas como VTune, gprof, Valgrind y cProfile son fundamentales en la industria.
- ☑ Medir primero, optimizar después es la única estrategia profesional.
- ☑ La optimización debe basarse en datos, no en intuición.

💬 *"El backtracking es poderoso, pero con gran poder viene gran responsabilidad computacional. Úsenlo sabiamente. Y recuerden: el profiling es su brújula en el mundo del rendimiento."*

12. Bibliografía y Recursos Recomendados

Bibliografía Complementaria

- Cormen, T. H. et al. (2022). *Introduction to Algorithms* (4ª ed.). MIT Press.
→ Capítulo 3: Growth of Functions.
- Jiménez Murillo, J. A. (2008). *Matemáticas para Computación*. Alfaomega.

Recursos en Línea

- Intel VTune Documentation
⇒ <https://www.intel.com/content/www/us/en/developer/tools/oneapi/vtune-profiler.html>
- Python cProfile Tutorial
⇒ <https://docs.python.org/3/library/profile.html>
- Valgrind – Callgrind and Cachegrind
⇒ <https://valgrind.org/docs/manual/cl-manual.html>
- Haskell Criterion
⇒ <https://hackage.haskell.org/package/criterion>